

PAF – INSTITUTE OF APPLIED SCIENCES AND TECHNOLOGY

Department of Computer Science & Artificial Intelligence

ASSIGNMENT

Hebbian/Neo-Hebbian Learning & Drive-Reinforcement Theory in ANNs

Course Title:	Artificial Neural Network
Course Code:	COMP-341
Instructor:	Dr. Abid Ali
Class	BS AI Red F23
Submission Date	26-02-2026
Deadline	05-03-2026

Course Learning Outcomes (CLOs) & Program Learning Outcomes (PLOs)

This assignment is designed to assess the following CLOs and their mapping to PLOs:

CLO #	Course Learning Outcome	Description	PLO #	Bloom's Level
CLO-1	Understand and explain Hebbian and Neo-Hebbian learning rules and their biological basis	Students will demonstrate theoretical knowledge of Hebb's postulate, LTP/LTD, and Oja's rule	PLO-1	C2 – Comprehension
CLO-2	Apply unsupervised learning algorithms (PCA, Oja's Rule) to real datasets using Python	Students implement and test principal component extraction using Hebbian methods in Jupyter Notebook	PLO-2	C3 – Application
CLO-3	Analyze biological and computational limitations of classical Hebbian learning	Students critically evaluate issues of saturation, temporal specificity, and predictive causality	PLO-3	C4 – Analysis

PLO #	Program Learning Outcome	ABET / HEC Category
PLO-1	Engineering Knowledge	Apply knowledge of mathematics, science, and computing fundamentals to neural network models
PLO-2	Problem Analysis & Design	Identify, formulate, and solve computing problems using principled design methods

PLO-3	Investigation & Critical Thinking	Conduct literature review, experiment design, and data analysis to support conclusions
PLO-4	Modern Tool Usage	Apply modern software tools (Python, NumPy, Matplotlib, Jupyter) to solve AI/ML problems

Marks Distribution

Question	Topic	Max Marks	Type
Q1	Hebbian Learning – Theoretical Analysis	20	Theory + Python
Q2	Oja's Rule & Unsupervised PCA	20	Python Notebook
Q3	Neo-Hebbian / Differential Hebbian	20	Analysis + Code
Q4	Drive-Reinforcement Theory (DRT)	20	Theory + Simulation
Q5	DRT vs. Temporal Difference Learning	20	Comparative Analysis
Total		100	

General Instructions

- Submit a single PDF of your written answers AND a separate Jupyter Notebook (.ipynb) file.
- All code must be original. Plagiarism will result in zero marks for the entire assignment.
- Comment every function and significant code block in your Python notebook.
- Label all plots with titles, axis labels, and legends.
- Ensure your notebook runs from top-to-bottom without errors before submission.
- Late submissions will be penalized 10% per day.

Question 1: Oja's Rule & Unsupervised PCA – Python Notebook [20 Marks]

CLO Assessed: CLO-2

PLO Assessed: PLO-2, PLO-4

Implement the following tasks in a Jupyter Notebook. Include all outputs, plots, and commentary.

Task 2.1 – Basic Hebbian Rule (Unstable) [5 Marks]

Implement the basic Hebbian learning rule and demonstrate its weight instability.

Required code structure (starter template):

```
import numpy as np
import matplotlib.pyplot as plt

def basic_hebbian(X, eta=0.01, n_epochs=50):
    """
    Basic Hebbian learning rule:  $\Delta w = \eta * x * y$ 
    X: input data (n_samples x n_features)
    """
    w = np.random.randn(X.shape[1]) * 0.01 # Initialize small random weights
    weight_norms = []
    for epoch in range(n_epochs):
        for x in X:
            y = np.dot(w, x) # Linear output
            w += eta * y * x # Hebbian update (UNSTABLE)
            weight_norms.append(np.linalg.norm(w))
    return w, weight_norms
```

Your Tasks:

- Generate a 2D dataset of 200 points from a multivariate normal distribution with a non-diagonal covariance matrix (e.g., strong correlation between variables).
- Run `basic_hebbian()` and plot the weight norm vs. epoch. Discuss the observed instability.
- Show that the weight vector does NOT converge to the principal component of the data.

Task 2.2 – Oja's Rule (Stable PCA) [10 Marks]

Implement Oja's rule, which adds a normalization term to stabilize Hebbian learning:

$$\Delta w = \eta \cdot y \cdot (x - y \cdot w)$$

- Complete the `ojas_rule()` function below and implement the normalization term.

```
def ojas_rule(X, eta=0.01, n_epochs=100):
    w = np.random.randn(X.shape[1]) * 0.01
    for epoch in range(n_epochs):
        for x in X:
            y = np.dot(w, x)
            # TODO: Implement Oja's update rule here
            # w += ...
    return w / np.linalg.norm(w) # Return unit vector
```

- Using the same dataset from Task 2.1, apply `ojas_rule()` and extract the first principal component.
- Use `sklearn.decomposition.PCA` to get the ground-truth first PC and compare the angle between the two vectors.
- Plot: (i) the data with both PCA eigenvector and Oja's learned vector overlaid, (ii) convergence of $\|w\|$ over epochs.

- Write a markdown cell explaining why Oja's rule is classified as a Neo-Hebbian rule.

Task 2.3 – Receptive Fields from Natural Image Patches [5 Marks]

- Download or generate 10,000 random 8x8 patches from a grayscale natural image (or use `sklearn.datasets.fetch_olivetti_faces`).
- Apply Oja's rule to learn the first 10 principal components (use a loop or extend the rule).
- Display the 10 learned weight vectors as 8x8 images. Compare their visual appearance to Gabor filters.
- In a markdown cell, explain the connection between Hebbian learning, PCA, and the emergence of Gabor-like receptive fields in V1.

Question 3: Differential Hebbian Learning & Neo-Hebbian Extensions [20 Marks]

CLO Assessed: CLO-2, CLO-3

PLO Assessed: PLO-2, PLO-3, PLO-4

Part A – Theory [8 Marks]

1. (a) Define Differential Hebbian Learning. How does it differ from standard Hebbian learning? Write the mathematical formulation using derivatives of input and output signals. [3 marks]
2. (b) Explain the concept of 'Spike-Timing Dependent Plasticity' (STDP). Describe the temporal window: what happens when pre-synaptic firing precedes post-synaptic firing vs. the reverse? [3 marks]
3. (c) How does the Mexican Hat correlation structure in Linsker's model explain the emergence of orientation-selective cells in V1? [2 marks]

Part B – Python: Simulating STDP [12 Marks]

Simulate a simplified version of Spike-Timing Dependent Plasticity.

- Create two spike trains (pre-synaptic and post-synaptic) as binary time series of length $T=1000\text{ms}$.
- Implement the STDP learning window function using the standard exponential model:
$$\Delta W = A_+ \cdot \exp(-|\Delta t| / \tau_+) \quad \text{if } \Delta t > 0 \quad (\text{pre before post} \rightarrow \text{LTP})$$
$$\Delta W = -A_- \cdot \exp(-|\Delta t| / \tau_-) \quad \text{if } \Delta t < 0 \quad (\text{post before pre} \rightarrow \text{LTD})$$
- Use parameters: $A_+ = 0.01$, $A_- = 0.012$, $\tau_+ = 20\text{ms}$, $\tau_- = 20\text{ms}$.
- Run the simulation for 50 epochs and plot: (i) the STDP learning window ΔW vs. Δt , (ii) the synaptic weight w over time.
- Repeat with a causal spike train (pre always fires 10ms before post) and an anti-causal train (post before pre). Compare the final weight changes.
- Write a markdown cell explaining how STDP implements a form of temporal Hebbian learning and connects to differential Hebbian learning.

Grading Rubric

Criterion	Excellent (Full Marks)	Satisfactory (Partial)	Poor (Low/No Marks)
Conceptual Accuracy	Precise, correct use of terminology; no factual errors	Minor errors or imprecise language but overall understanding shown	Fundamental misconceptions or mostly incorrect
Python Code Quality	Code runs, is well-commented, modular, and efficient	Code runs with minor issues; some comments missing	Code fails to run or is largely incomplete
Plot Quality	All plots labelled, titled, and clearly interpreted	Plots present but missing labels or interpretation	Plots missing or unreadable
Mathematical Notation	Correct and consistent use of notation throughout	Mostly correct with minor errors	Incorrect or absent mathematical notation
Written Explanations	Clear, precise, and demonstrates deep understanding	Adequate but superficial; lacks depth	Missing or incoherent

Academic Integrity & Submission Policy

Academic Integrity Notice: Submitting work that is not your own constitutes academic misconduct. All code will be checked with plagiarism detection tools. Any student found submitting copied work will receive zero marks for the assignment and may face further disciplinary action as per university policy.

Submission Format:

- File 1: Assignment_RollNumber.pdf — All written answers, derivations, and plot screenshots.
- File 2: Assignment_RollNumber.ipynb — Fully executed Jupyter Notebook (all outputs visible).
- Upload both files to the CMS before the deadline.

— End of Assignment —

Dr. Abid Ali | abid.ali@paf-iastr.edu.pk | PAF-IASTR